

工业视觉流程编辑器

代码解析与开发维护手册

基线版本：FastAPI 应用 2.2.0

分析对象：industrial_vision_refactored_error_trace_v2.zip

SHA-256：f4be35dbfda49c7b5bd11880eedc72e9a00b33f0a659e6b1e13dd372bc2aca7a

用途：架构理解、功能扩展、算子开发、故障排查与版本交付

生成日期：2026-06-25

使用说明

本手册针对当前可运行基线版本进行静态代码阅读、实际导入、接口验证和自动化测试。内容既描述“代码现在如何工作”，也标注了扩展点、约束、已知风险和建议的重构方向。后续开发时，应把本手册与源代码、测试用例共同维护。

验证状态：当前基线执行 `pytest -q`，30 项测试全部通过。项目总计约 5,979 行 Python/JavaScript/CSS/HTML，其中 `frontend/assets/js/app.js` 约 1,733 行，是当前最大的维护热点。

目录

- 1. 项目定位与能力边界
- 2. 启动、依赖与运行方式
- 3. 目录结构和模块职责
- 4. 系统架构与请求链路
- 5. 工作流数据模型与端口协议
- 6. 算子系统：实现、元数据、注册和插件
- 7. 单图工作流执行器
- 8. 批量处理与 CSV 汇总
- 9. 逻辑节点语义与限制
- 10. 前端架构与交互状态
- 11. API 通信协议
- 12. AST 独立算法脚本导出
- 13. 自定义算子开发指南
- 14. 错误处理与排障
- 15. 自动化测试与交付检查
- 16. 已知问题、技术债与演进建议
- 附录 A：算子清单
- 附录 B：关键函数索引
- 附录 C：常用修改路径与检查清单

1. 项目定位与能力边界

该项目是一个基于 FastAPI、OpenCV、NumPy 和原生 Web 技术构建的节点式工业视觉流程编辑器。用户通过浏览器拖拽图像节点和逻辑节点、连接端口、修改参数，再由后端按有向无环图（DAG）的拓扑顺序执行。

- 单图流程：从服务器可访问的图片路径读取图像，逐节点执行并返回每一步预览。
- 批量流程：枚举目录中的图片，逐张运行同一工作流，保存结果图并汇总 CSV。
- 图像算子：颜色、增强、滤波、阈值、边缘、形态学、几何变换和轮廓分析。
- 逻辑算子：常量、比较、布尔运算、If/Else 数据选择、数值 For/While 循环。
- 插件机制：自动扫描 backend/custom_operators 下的自定义算子。
- 脚本导出：将可支持的工作流通过 AST 编译为独立运行的普通 Python/OpenCV 脚本。
- 多工作流前端：标签页、Ctrl+S 本地保存、JSON 导入导出、画布缩放和平移、导航视图。
- 节点级错误追踪：失败时保留已完成节点预览，标记失败节点和未执行节点。

重要边界：系统仍是 DAG 数据流执行器。If/Else 不会跳过未选择分支的上游计算；For/While 仅做安全的数值迭代，不是可包含任意子流程的控制流容器。

2. 启动、依赖与运行方式

2.1 启动入口

项目根目录的 run.py 使用 Uvicorn 加载 backend.main:app，监听 127.0.0.1:8000，并开启 reload=True。reload 适合开发环境，不适合生产部署。

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate
pip install -r requirements.txt
python run.py
```

浏览器访问 <http://127.0.0.1:8000>。FastAPI 在 /assets 挂载静态资源，根路径返回 frontend/index.html。

2.2 依赖

依赖	职责
fastapi	HTTP API、依赖注入、Pydantic 集成
uvicorn[standard]	ASGI 服务器
python-multipart	兼容的图片上传接口
numpy	图像数组和数值计算
opencv-python	图像处理与编码/解码
pydantic	工作流请求模型验证
pytest	自动化测试

2.3 公司环境运行建议

- 必须从包含 run.py 的项目根目录启动，避免 backend 模块解析错误。
- 不要直接打开 index.html，也不要只使用静态服务器；前端需要同源 /api 接口。
- 部署到其他机器访问时，将 host 改为 0.0.0.0，并通过防火墙和反向代理控制访问。
- 生产环境关闭 reload，建议使用多 worker 或进程管理器，并限制服务器文件路径访问范围。

3. 目录结构和模块职责

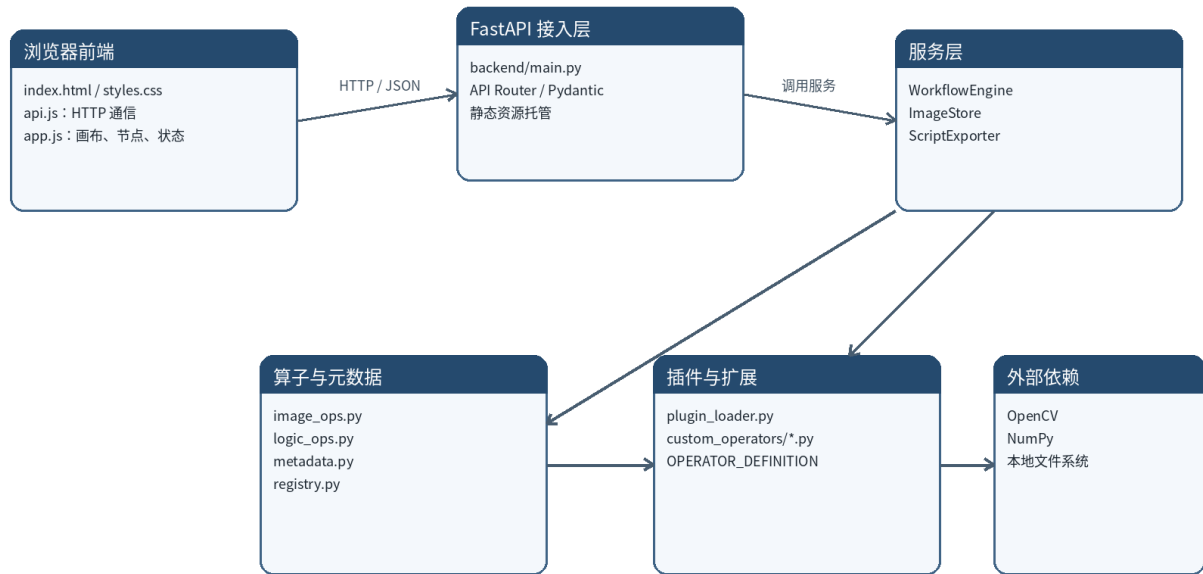
```

industrial_vision_refactored_error_trace_v2/
├── run.py
├── requirements.txt
├── backend/
│   ├── main.py / config.py
│   ├── api/                # FastAPI 路由与依赖
│   ├── models/             # Pydantic 工作流模型
│   ├── algorithms/         # 图像/逻辑算子、元数据、注册表、插件加载
│   ├── custom_operators/   # 用户自定义插件
│   └── services/           # 工作流执行、路径存储、AST 导出
├── frontend/
│   ├── index.html
│   └── assets/css, assets/js
├── examples/               # 批量/逻辑工作流示例
├── docs/                   # 自定义算子说明
├── tests/                  # 30 项自动化测试
└── uploads/                # 旧上传 API 的兼容目录
  
```

文件/目录	核心职责
run.py	开发启动入口；加载 backend.main:app。
backend/main.py	创建 FastAPI 应用、注册 API、挂载静态资源、构造 ImageStore。
backend/config.py	项目根目录、前端目录、上传目录、扩展名和上传大小。
backend/api/routes/*	HTTP 协议适配：算子列表、运行、导出、旧上传。
backend/models/workflow.py	请求结构和数量上限。
backend/algorithms/image_ops.py	内置 OpenCV 图像算子。
backend/algorithms/logic_ops.py	常量、比较、布尔、选择和安全数值循环。
backend/algorithms/metadata.py	前端节点标签、分组、端口、参数和结果展示声明。
backend/algorithms/registry.py	把类型名映射到执行函数，并合并自定义插件。
backend/algorithms/plugin_loader.py	扫描、导入、校验 custom_operators。
backend/services/workflow_engine.py	拓扑排序、节点执行、预览、批量、CSV、错误现场。
backend/services/image_store.py	路径解析、中文路径读取、目录枚举和输出目录创建。
backend/services/script_exporter.py	AST 编译独立算法脚本。
frontend/assets/js/api.js	统一封装 fetch 和错误响应。
frontend/assets/js/app.js	前端状态、画布、节点、连线、预览、标签和导航。
tests/*	覆盖后端执行、批量、脚本导出、逻辑节点和前端结构。

4. 系统架构与请求链路

系统分层架构



核心原则：路由只处理协议，服务层负责编排，算子只处理单节点计算。

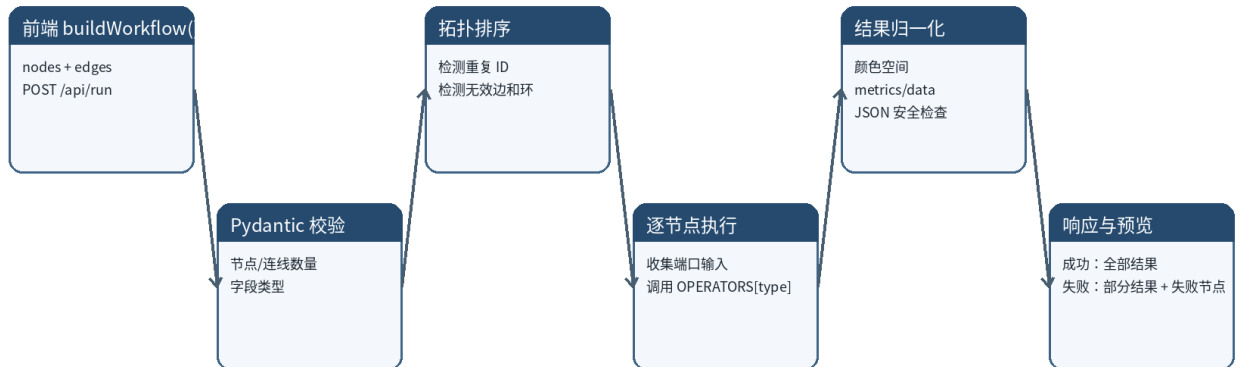
图 1 系统分层架构

4.1 分层原则

- 前端只通过 VisionApi 调用后端，不直接耦合 fetch 细节。
- API 路由只做请求/响应转换，核心业务放在 services。
- 工作流引擎不实现具体算法，只通过 OPERATORS 注册表调用算子。
- 算子只处理当前节点输入、参数和 ExecutionContext，不理解整个画布。
- 元数据驱动前端节点展示，执行函数与 UI 描述分离。

4.2 单图执行顺序

单图 workflow 执行链路



异常策略：算子内部或结果序列化失败时，WorkflowExecutionError 携带 executed_order、previews、failed_node 和 pending_nodes。

图 2 单图 workflow 执行链路

拓扑顺序由节点列表和边共同决定：所有入度为 0 的节点先进入队列；并行分支之间的先后顺序主要受 JSON 中节点顺序影响，而不是画布的 x 坐标。

5. workflow 数据模型与端口协议

5.1 Pydantic 模型

模型	字段	说明
NodePosition	x, y	画布世界坐标。
WorkflowNode	id, type, position, params, collapsed	节点 ID/类型、位置、参数和折叠状态；单次最多 500 个。
WorkflowEdge	source, target, sourceHandle, targetHandle	连接指定输出端口到指定输入端口；单次最多 2000 条。
WorkflowRequest	version, image_id, nodes, edges	完整运行或导出请求；image_id 仅兼容旧上传流程。

```
{
  "version": "2.1",
  "nodes": [
    {
      "id": "n1",
      "type": "ReadImage",
      "position": {
        "x": 100,
        "y": 100
      },
      "params": {
        "image_path": "samples/input.png"
      },
      "collapsed": false
    }
  ],
  "edges": []
}
```

5.2 算子输入/输出字典

每个算子函数统一接收 inputs、params、context，返回字典。返回字典中的键同时承担“输出端口”和“系统保留字段”两类职责。

键	语义
out	默认输出端口；可为 np.ndarray，也可为数值、布尔或文本。
任意自定义键	独立输出端口，例如 mean_brightness、brightness_std。
<handle>_color_space	对应图像输出端口的颜色空间；常用 BGR/RGB/GRAY。
metrics	当前节点自己的可序列化指标；引擎会移入 data，并用于预览/CSV。
data	显式的累积数据字典；会与上游 data 合并并继续传递。

JSON 安全约束：metrics 和 data 只能包含 None、字符串、数字、布尔、字典、列表等可序列化数据。NumPy 标量会转换为 Python 标量；np.ndarray 必须从图像输出端口传递，不能放入 metrics/data。

5.3 元数据驱动的端口

metadata 中的 inputs/outputs 决定前端端口数量、标签和类型。没有显式 inputs/outputs 的旧图像节点使用默认 in/out。

```
"inputs": [
  {"handle": "in", "label": "图像", "type": "image", "required": true}
],
"outputs": [
  {"handle": "out", "label": "图像", "type": "image"},
  {"handle": "mean_brightness", "label": "平均亮度", "type": "number"}
]
```

端口类型主要由前端用于连线提示。后端会检查必需端口是否存在，但尚未形成统一的后端类型系统；导入恶意或手写 JSON 时，仍可能在算子运行阶段才发现类型错误。

6. 算子系统：实现、元数据、注册和插件

6.1 执行函数接口

```
def op_xxx(inputs, params, context):
    image = inputs["in"]
    result = ...
    return {
        "out": result,
        "out_color_space": "BGR",
        "metrics": {"score": 0.95},
    }
```

ExecutionContext 提供 image_store、当前批量图片路径、源目录、批次索引和总数。算子不应直接读取全局工作流，也不应修改全局状态。

6.2 内置图像算子

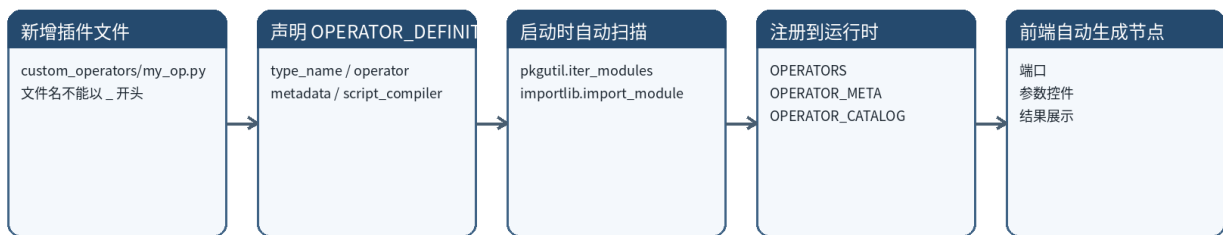
image_ops.py 中的算子尽量保持“一函数一操作”。helpers.py 提供宽松的参数转换、奇数核尺寸和颜色转换辅助。gray_input 会根据 in_color_space 正确处理 RGB/BGR。

6.3 注册表

registry.py 的 OPERATORS 将节点 type 映射到函数；OPERATOR_META 和 OPERATOR_CATALOG 决定 UI；自定义插件在模块导入时被扫描并加入三个结构。若 type_name 与内置类型冲突，应用启动失败。

6.4 插件加载

自定义算子加载与注册



核心返回约定：out/独立输出端口传递数据；out_color_space 标注颜色空间；metrics 用于界面和批量 CSV；data 用于累积传递。

注意：一个插件导入失败会阻止服务启动；生产环境建议增加隔离加载和错误列表。

图 3 自定义算子加载与注册

plugin_loader.py 使用 pkgutil.iter_modules 扫描 backend.custom_operators。以下划线开头的模块被忽略；其他模块必须导出 OPERATOR_DEFINITION 字典。加载发生在 registry.py 导入阶段。

维护建议：当前插件加载是“失败即阻止启动”。公司环境中建议改为逐插件 try/except，记录 failed_plugins，并让 /api/operators 返回插件错误列表，避免单个实验插件导致整个页面不可用。

7. 单图 workflow 执行器

7.1 topological_sort

- 拒绝重复节点 ID。
- 拒绝 source/target 不存在的边。
- 使用 Kahn 算法计算顺序。
- 若排序结果数量少于节点数量，判定存在环。

7.2 _execute_once 核心步骤

1. 根据 incoming 边收集每个 targetHandle 的输入值。
2. 同步传播 <handle>_color_space 和上游 data。

3. 根据 `metadata.inputs` 校验必需端口；旧节点使用 `has_input`。
4. 调用 `OPERATORS[node.type](inputs, params, context)`。
5. 补全默认颜色空间；校验 `result`、`metrics`、`data`。
6. 把当前 `metrics` 存入 `data` 的“节点 ID_节点名称”键。
7. 生成图像或数值预览；记录 `executed_order`。
8. 发生异常时封装 `WorkflowExecutionError`，保留已完成预览。

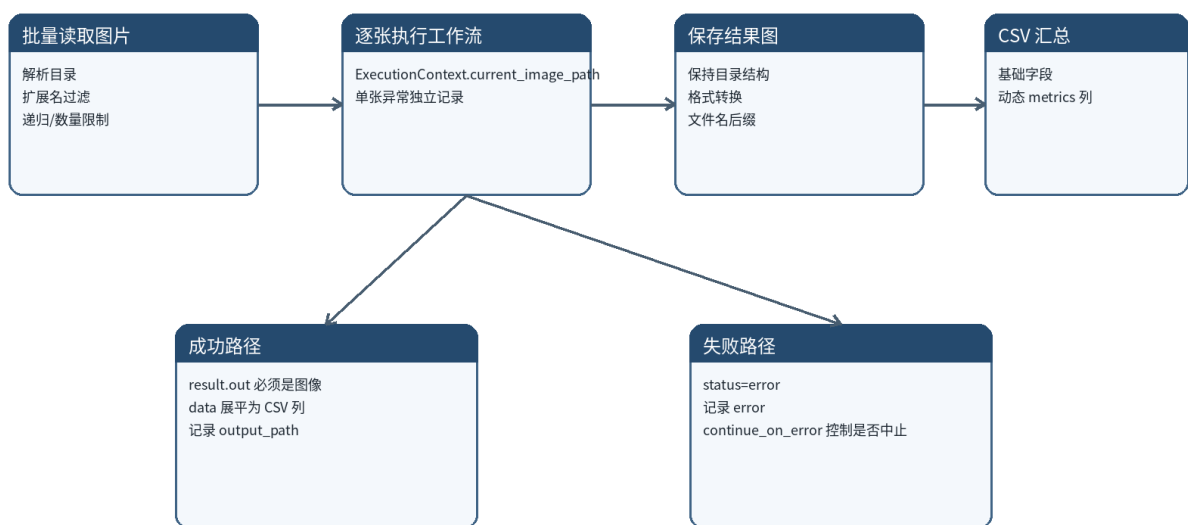
7.3 失败响应

```
{
  "success": false,
  "stage": "node_execution",
  "executed_order": ["n1", "n2"],
  "pending_nodes": ["n4"],
  "failed_node": {
    "node_id": "n3",
    "type": "LogicCompare",
    "label": "比较",
    "error": "节点 n3 ... 执行失败: ..."
  },
  "previews": [/* 已完成节点 + 错误节点 */]
}
```

前端 `api.js` 会把非 2xx 响应转换成 `Error`，并把原始 `payload` 挂到 `error.payload`。`runWorkflow` 捕获后仍调用 `renderPreviews(payload.previews)`，因此已完成结果不会丢失。

8. 批量处理与 csv 汇总

批量处理与 CSV 汇总



批次结束后，再对最后一张成功图片执行一次预览，以避免向浏览器返回大量 Base64 图片。

图 4 批量处理与 CSV 汇总

8.1 批量流程结构约束

- 必须且只能有一个 BatchReadImages。
- 不能同时包含 ReadImage。
- 必须且只能有一个 SaveBatchResults。
- 批量输入不能有上游连线。
- 保存节点必须只有一个上游，并且必须是终点。

8.2 每张图片的执行

目录枚举由 ImageStore.list_images 完成。每张图片创建独立 ExecutionContext，并调用同一个 _execute_once。成功时保存 saver.out，并把 saver.data 展平成 CSV 列；失败时写 status/error。continue_on_error 决定是否继续。

8.3 CSV 结构

固定列	含义
source_path	输入图片绝对路径
source_name	输入文件名
relative_path	相对源目录路径
output_path	结果图片路径
status	success/error
error	单张失败错误

动态指标列来自 data 的递归展平，例如 n5_缺陷检测.defect_count。列表会 JSON 编码。CSV 使用 utf-8-sig，便于 Excel 直接打开中文。

性能特征：批量运行是同步 HTTP 请求，不提供进度流；大目录可能导致请求持续很久。完成后还会对最后一张成功图片额外执行一次，用于节点预览。

9. 逻辑节点语义与限制

类型	输入	运行语义
LogicConstant	无输入	输出数值/布尔/文本常量。
LogicCompare	a, b	GT/GE/LT/LE 强制标量数值；EQ/NE 支持标量文本/布尔。
LogicBoolean	a, b	AND/OR/XOR/NAND/NOR。
LogicNot	in	布尔取反。
LogicIfElse	condition, true_value, false_value	从已计算的两个候选结果中选择一个。
LogicForLoop	可选 in	固定次数执行加减乘除，最多 10000 次，history 最多 100 项。
LogicWhileLoop	可选 in	条件迭代，强制 max_iterations 1..10000。

If/Else 不是惰性分支：由于工作流是 DAG，true_value 和 false_value 的上游节点在 If/Else 执行前都已经运行。当前节点只是选择数据，不是传统编程语言中的控制流跳转。

循环不是子流程容器：For/While 只在节点内部更新一个数值，不能让一组图像节点循环执行。若要支持真正的循环体，需要引入子图、作用域和控制令牌，属于执行器架构升级。

10. 前端架构与交互状态

10.1 页面组成

区域	功能
顶部标签与操作栏	多工作流、JSON 导入导出、示例、运行、清空。
左侧 rail	节点库显示/隐藏、导出算法脚本、占位功能。
节点库	图像与逻辑库，搜索、分类展开/折叠、拖拽。
中心画布	节点、端口、连线、平移缩放、导航视图。
右侧面板	状态、节点/连线信息、每步运行预览。
Lightbox	原图查看与下载。

10.2 全局状态

变量	职责
nodes / edges	当前活动工作流的节点与连线。
selectedNodeId / selectedEdgeId	当前选择。
pan / zoom	无限画布视图。
nodePreviews	Map：按 node_id 缓存运行结果。
workflowDocuments	多标签文档集合。
activeWorkflowId	活动标签。
collapsedOperatorGroups	节点库分类折叠状态。
minimapTransform	世界坐标和导航视图像素之间的映射。

10.3 多工作流持久化

前端把多个工作流、保存快照、标签名称、画布视图和选择状态保存到 localStorage，键名为 vision_python_mvp_workspace_v1。Ctrl+S 更新当前文档 savedSnapshot；白点表示已保存，橙点表示 dirty。该保存只在当前浏览器本地，不会写入后端。

10.4 元数据驱动的患者渲染

renderNodes 根据 OPERATOR_META 生成标题、端口、参数控件和结果区。param_schema 控制 number/select/text 控件；没有 schema 时使用参数类型推断。多输出端口由 outputs 数组生成。

10.5 型兼容

前端 findPortDefinition 和 portsAreCompatible 在创建连线前检查类型。any 可连接任意类型；相同类型可连接。每个输入端口只保留一条连线，finishConnect 会删除同一 target/targetHandle 的旧边。

10.6 预览与错误展示

renderPreviews 同时更新右侧列表和 nodePreviews Map；renderNodePreviewHtml 根据 result_type/status 选择图像、数值、混合或错误视图。预览图片可下载和打开 Lightbox。

10.7 导航视图

computeMinimapTransform 使用所有节点矩形和当前视口计算动态边界、缩放和偏移；点击导航视图时将像素反算为世界坐标，再调整 pan，使目标点位于主视口中心。

主要维护热点：app.js 约 1,733 行，包含文档管理、画布、节点、连线、参数、预览、批量摘要、下载、导航和快捷键。功能继续增长前，应拆成 workspace.js、canvas.js、nodes.js、preview.js、storage.js 等模块。

11. API 通信协议

方法	路径	说明
GET	/api/operators	返回 operators 元数据和 catalog 分类。
POST	/api/run	自动识别单图或批量流程；成功或结构化失败响应。
POST	/api/export-script	返回 text/x-python 附件 vision_algorithm.py。
POST	/api/upload	旧版兼容上传，限制 20 MB；当前前端不使用。

11.1 api.js 错误策略

request() 首先尝试解析 JSON。响应不是 JSON 时抛出“服务返回了无法解析的响应”；HTTP 非成功或 data.success=false 时抛出 Error，并保留 payload 和 httpStatus。这个设计是节点级错误现场能够传到 app.js 的关键。

11.2 同源部署假设

API_BASE 固定为 /api，静态资源路径固定为 /assets。当前设计假设页面和后端同源并部署在站点根路径。如果部署到 /vision/ 子路径，需要引入可配置 base URL 或调整反向代理。

12. AST 独立算法脚本导出

12.1 生成流程

1. 复用 topological_sort 得到执行顺序。
2. 把 Gray/BGR2RGB 迁移为 ColorConvert，把批量输入/保存映射为单图输入/终点。
3. 为每个节点生成普通 Python 变量。

4. 内置图像算子由 `_compile_operator` 转换为 OpenCV 表达式。
5. 逻辑节点由 `_compile_logic_operator` 生成条件表达式、for 或 while AST。
6. 拼接 `run_algorithm`、读写图片和命令行 `main`。
7. `ast.unparse` 输出源码，`compile` 再次校验语法。
8. 检查源码中不得出现 WORKFLOW、OPERATORS、nodes/edges 等运行时结构。

12.2 导出脚本运行方式

```
pip install opencv-python numpy
python vision_algorithm.py --input input.png --output result.png
```

12.3 当前重要限制

多输出端口导出缺陷：当前 exporter 在处理边时只按“源节点”映射变量，没有根据 sourceHandle 选择独立输出。MeanBrightness.mean_brightness 连接 LogicCompare 时，导出脚本仍会把 MeanBrightness 的图像变量传给比较节点。网页运行正确，但该类多输出工作流导出的脚本语义错误。建议把 variables 改为以 (node_id, output_handle) 为键，并让自定义 script_compiler 返回各输出端口表达式。

- 自定义 script_compiler 当前接口只返回一个主表达式和颜色空间，不足以表达多个输出端口。
- 导出器要求恰好一个 ReadImage/BatchReadImages。
- 图像节点要求恰好一个 in 输入；复杂多输入图像算法需要扩展编译协议。
- 批量流程导出为单图 run_algorithm，不导出目录遍历与 CSV。

13. 自定义算子开发指南

13.1 最小步骤

1. 复制 backend/custom_operators/_template.py。
2. 重命名为不以下划线开头的文件。
3. 实现 `op_xxx(inputs, params, context)`。
4. 定义 OPERATOR_DEFINITION。
5. 重启服务并访问 `/api/operators` 检查注册。
6. 增加单元测试和批量/脚本导出测试。

13.2 OPERATOR_DEFINITION 字段

字段	说明
type_name	全局唯一的节点类型名。
operator	可调用的执行函数。
metadata.label	前端显示名。
metadata.kind	节点视觉类型，如 measure/custom/logic。
metadata.inputs/outputs	多端口定义。

字段	说明
metadata.params	默认参数。
metadata.param_schema	控件类型、标签、选项、提示。
metadata.result_type	metrics / auto；控制主要预览类型。
category/group	左侧节点库分类。
script_compiler	可选；独立算法脚本编译器。

13.3 推返回模式

```
return {
    "out": output_image,
    "out_color_space": inputs.get("in_color_space", "BGR"),
    "score": float(score),          # 独立数值输出端口
    "metrics": {
        "score": float(score),
        "defect_count": int(count),
    },
}
```

- OpenCV/NumPy 计算结果必须转换成 Python int/float/bool 后放入 metrics。
- 输出图像必须是 np.ndarray，并通过 out 或其他明确的图像端口传递。
- 多输出时 metadata.outputs 必须与返回字典键一致。
- 批量 CSV 只会收集沿工作流传到保存节点的 data。
- 不要在算子中维护跨请求的可变全局状态。

13.4 核心算子与插件算子的选择

业务专用、可独立启停的算法优先放 custom_operators；平台基础能力放 image_ops/logic_ops。修改核心算子通常还要同步 metadata、registry、script_exporter 和测试，插件则可通过单文件完成运行时扩展。

14. 错误处理与排障

现象	排查方向
页面只有静态外壳	确认 Network 中 api.js/app.js 是否请求；查看 view-source；检查 index.html 与静态文件版本。
/api/operators 正常但节点库为空	查看 Console 首个错误；常见原因是 app.js 与 index.html 版本不一致或 DOM 元素为空。
HTTP 500 且无节点预览	检查是否有未捕获异常或响应序列化问题；当前 _json_safe_value 已防止数组混入 metrics/data。
某节点缺少输入	检查 metadata.inputs.required、targetHandle 和是否被另一条边覆盖。
比较节点提示收到图像	应连接测量算子的数值输出端口，而不是 out 图像端口。
图片路径不可访问	路径是后端机器路径；确认扩展名、权限和中文路径。
自定义算子导致服务启动失败	临时把插件文件改成下划线开头；检查依赖和 OPERATOR_DEFINITION。
批量没有 CSV 指标	确保测量算子的 metrics 沿数据链传到 SaveBatchResults。
导出脚本结果与网页不一致	重点检查多输出端口、颜色空间和自定义 script_compiler。

14.1 推荐日志增强

- 为每次运行生成 request_id，并记录 node_id、耗时、输入输出类型和图像尺寸。
- 使用 logging 替代仅返回错误字符串；开发和生产配置不同级别。
- 批量任务记录进度和每张耗时。
- 插件加载失败写入独立错误清单。

15. 自动化测试与交付检查

15.1 当前测试覆盖

测试文件	覆盖重点
test_workflow_engine.py	示例流程、环检测、数值/图像预览、节点失败部分结果。
test_logic_nodes.py	逻辑注册、If/Else、循环、AST 导出、API 结构化错误、多输出数值端口。
test_batch_workflow.py	插件自动注册、批量保存、CSV 多指标。
test_script_exporter.py	独立算法脚本生成和运行、下载文件名。
test_frontend_layout.py	页面结构、节点工具栏、预览、标签、节点库、导航和错误渲染代码。

```
pytest -q
# 当前基线: 30 passed
```

15.2 每次交付的最低检查

1. 从确认的完整基线复制后再修改，不在残缺目录上叠加。
2. 运行 Python 编译检查和 pytest。
3. 检查 JavaScript 语法。
4. 实际启动 python run.py。
5. 访问 /、/assets/js/app.js、/api/operators。
6. 运行一个单图流程、一个错误流程和一个批量流程。
7. 导出并在独立目录运行 vision_algorithm.py。
8. 检查 ZIP 中包含 run.py、requirements.txt、完整 backend/frontend/tests。
9. 用新目录解压 ZIP 后再次启动，避免打包路径错误。

16. 已知问题、技术债与演进建议

优先级	问题	建议
高	脚本导出不支持多输出端口	重构变量表为 (node_id, handle)，扩展编译器返回 outputs 映射，并增加 MeanBrightness→Compare 导出测试。
高	服务器路径访问范围过宽	为 ImageStore 配置允许的输入根目录和输出根目录，拒绝越界绝对路径；网络部署必须鉴权。

优先级	问题	建议
高	批量任务同步执行	引入任务队列、job_id、进度接口或 WebSocket，防止长请求超时。
中	app.js 单文件过大	拆分 ES Modules: workspace、canvas、nodes、edges、preview、minimap、storage。
中	后端端口类型校验不统一	建立类型系统和边校验器，运行前检查 image/number/boolean/any。
中	插件错误会阻止启动	隔离加载，允许部分插件失败，并在 UI 展示错误。
中	每节点返回完整 Base64 图片	生成缩略图或缓存 URL；只在需要时请求原图。
中	本地存储无服务端版本管理	增加工作流 CRUD、数据库、版本号和并发控制。
低	旧上传 API 仍保留	明确废弃周期或重新接入受控文件选择。
低	run.py 默认 reload=True	区分 development/production 配置。

16.1 推荐演进顺序

1. 先修复 AST 多输出端口，这是当前网页运行与导出结果不一致的功能性缺陷。
2. 增加路径沙箱和身份认证，确保公司网络部署安全。
3. 拆分前端 app.js，并给核心模块增加单元测试。
4. 建立后端端口类型校验和工作流预校验接口。
5. 把批量执行改为异步任务，增加进度和取消。
6. 最后再评估真正的控制流子图、循环体和条件跳过。

附录 A：算子清单

A.1 图像节点

输入输出

类型	显示名	输入	输出	默认参数	用途
ReadImage	读取图像	无	out:image	image_path=	从服务器可访问的文件路径读取图片。
Display	结果显示	in:image	out:image	无	显示并传递上游结果。

批量处理

类型	显示名	输入	输出	默认参数	用途
BatchReadImages	批量读取图片	无	out:image	folder_path=, recursive=YES, extensions=,jpg,,jpeg,,png,.bmp,.tif,.tiff,.webp, max_images=0, continue_on_error=Y ES	枚举文件夹内图片，并依次把每张图片送入后续算法节点。

类型	显示名	输入	输出	默认参数	用途
SaveBatchResults	保存批量结果	in:image	无	output_dir=batch_outputs, save_images=YES, image_format=SOURCE, filename_suffix=_result, preserve_structure=YES, write_csv=YES, csv_filename=results.csv	保存每张处理结果图，并把源路径、文件名及所有数据结果汇总到 CSV。

颜色与增强

类型	显示名	输入	输出	默认参数	用途
ColorConvert	颜色转换	in:image	out:image	mode=GRAY	通过参数选择灰度化或 BGR 转 RGB。
Invert	反相	in:image	out:image	无	像素取反。
Normalize	归一化	in:image	out:image	无	拉伸到 0-255。
EqualizeHist	直方图均衡	in:image	out:image	无	灰度直方图均衡化。
CLAHE	CLAHE 增强	in:image	out:image	clip_limit=2.0, tile_grid=8	局部对比度增强。

滤波降噪

类型	显示名	输入	输出	默认参数	用途
MeanBlur	均值滤波	in:image	out:image	ksize=5	均值平滑。
GaussianBlur	高斯滤波	in:image	out:image	ksize=5, sigma=0	高斯平滑，常用于边缘检测前降噪。
MedianBlur	中值滤波	in:image	out:image	ksize=5	适合去椒盐噪声。
BilateralFilter	双边滤波	in:image	out:image	d=7, sigma_color=75, sigma_space=75	保边去噪。
Sharpen	锐化	in:image	out:image	amount=1.0	增强边缘与细节。

阈值与边缘

类型	显示名	输入	输出	默认参数	用途
Threshold	固定阈值	in:image	out:image	threshold=127, max_value=255	灰度二值化。
OtsuThreshold	Otsu 阈值	in:image	out:image	无	自动阈值分割。
AdaptiveThreshold	自适应阈值	in:image	out:image	block_size=11, c=2	适合光照不均场景。
Canny	Canny 边缘	in:image	out:image	threshold1=50, threshold2=150	经典边缘检测。
Sobel	Sobel 梯度	in:image	out:image	dx=1, dy=0, ksize=3	一阶梯度边缘。
Laplacian	Laplacian 边缘	in:image	out:image	ksize=3	二阶边缘检测。

形态学

类型	显示名	输入	输出	默认参数	用途
Erode	腐蚀	in:image	out:image	ksize=3, iterations=1	收缩白色区域。
Dilate	膨胀	in:image	out:image	ksize=3, iterations=1	扩张白色区域。
MorphOpen	开运算	in:image	out:image	ksize=3	先腐蚀后膨胀，常用于去小噪点。
MorphClose	闭运算	in:image	out:image	ksize=3	先膨胀后腐蚀，常用于补小孔。
MorphGradient	形态学梯度	in:image	out:image	ksize=3	提取形态学边缘。

几何变换

类型	显示名	输入	输出	默认参数	用途
Resize	缩放	in:image	out:image	scale_percent=50	按百分比缩放图像。
Flip	翻转	in:image	out:image	flip_code=1	1 水平, 0 垂直, -1 双向。
Rotate90	旋转 90 度	in:image	out:image	direction=0	0 顺时针, 1 逆时针, 2 旋转 180。

测量分析

类型	显示名	输入	输出	默认参数	用途
FindContours	轮廓查找	in:image	out:image	threshold=127, min_area=20	二值化后查找轮廓并绘制包围框。

自定义算子

类型	显示名	输入	输出	默认参数	用途
MeanBrightness	平均亮度统计	in:image	out:image, mean_brightness:number, brightness_std:number	无	计算图像平均亮度和标准差; 批量运行时自动写入 CSV。

A.2 逻辑节点

数据与变量

类型	显示名	输入	输出	默认参数	用途
LogicConstant	常量	无	out:any	value_type=NUMBER, value=0	产生数值、布尔值或文本常量, 供其他逻辑节点使用。

条件判断

类型	显示名	输入	输出	默认参数	用途
LogicCompare	比较	a:number, b:number	out:boolean	operator=GT	比较两个输入值并输出布尔结果。
LogicBoolean	布尔运算	a:boolean, b:boolean	out:boolean	operation=AND	对两个布尔条件执行与、或、异或等运算。
LogicNot	NOT 取反	in:boolean	out:boolean	无	对输入条件执行逻辑取反。

流程选择

类型	显示名	输入	输出	默认参数	用途
LogicIfElse	If / Else 选择	condition:boolean, true_value:any, false_value:any	out:any	无	根据条件从 If 和 Else 两个输入中选择一个结果。两个上游分支会先完成计算。

循环控制

类型	显示名	输入	输出	默认参数	用途
LogicForLoop	For 循环	in:number	out:number	initial_value=0, iterations=5, update_operation=A DD, step=1	按固定次数更新一个 数值，最多执行 10000 次。
LogicWhileLoop	While 循环	in:number	out:number	initial_value=0, compare_operator= LT, compare_value=10, update_operation=A DD, step=1, max_iterations=100	条件成立时持续更新 数值，并受最大循环 次数保护。

附录 B：关键函数索引

下表用于快速定位代码。行号基于本手册分析的基线版本，后续修改后可能发生偏移。

文件	函数/类	行	职责/文档
backend/algorithms/helpers.py	to_int	6	
backend/algorithms/helpers.py	to_float	12	
backend/algorithms/helpers.py	ensure_odd	18	
backend/algorithms/helpers.py	ensure_gray	26	
backend/algorithms/helpers.py	ensure_bgr	33	
backend/algorithms/helpers.py	kernel_from_params	40	
backend/algorithms/ image_ops.py	gray_input	17	
backend/algorithms/ image_ops.py	op_read_image	25	
backend/algorithms/ image_ops.py	op_batch_read_images	47	批量运行时读取当前图片。目录枚 举由工作流引擎负责。
backend/algorithms/ image_ops.py	op_save_batch_results	57	批量结果终点。实际写盘和 CSV 汇 总由工作流引擎统一完成。
backend/algorithms/ image_ops.py	op_display	64	
backend/algorithms/ image_ops.py	op_color_convert	71	
backend/algorithms/ image_ops.py	op_gray	97	旧版 Gray 节点兼容入口。
backend/algorithms/ image_ops.py	op_bgr_to_rgb	102	旧版 BGR2RGB 节点兼容入口。
backend/algorithms/ image_ops.py	op_invert	106	
backend/algorithms/ image_ops.py	op_normalize	109	
backend/algorithms/ image_ops.py	op_equalize_hist	112	
backend/algorithms/ image_ops.py	op_clahe	116	
backend/algorithms/ image_ops.py	op_mean_blur	123	
backend/algorithms/ image_ops.py	op_gaussian_blur	127	
backend/algorithms/ image_ops.py	op_median_blur	132	
backend/algorithms/ image_ops.py	op_bilateral_filter	136	
backend/algorithms/ image_ops.py	op_sharpen	142	
backend/algorithms/ image_ops.py	op_threshold	148	

文件	函数/类	行	职责/文档
backend/algorithms/image_ops.py	op_otsu_threshold	155	
backend/algorithms/image_ops.py	op_adaptive_threshold	160	
backend/algorithms/image_ops.py	op_canny	167	
backend/algorithms/image_ops.py	op_sobel	173	
backend/algorithms/image_ops.py	op_laplacian	181	
backend/algorithms/image_ops.py	op_erode	187	
backend/algorithms/image_ops.py	op_dilate	191	
backend/algorithms/image_ops.py	op_morph_open	195	
backend/algorithms/image_ops.py	op_morph_close	198	
backend/algorithms/image_ops.py	op_morph_gradient	201	
backend/algorithms/image_ops.py	op_resize	204	
backend/algorithms/image_ops.py	op_flip	212	
backend/algorithms/image_ops.py	op_rotate90	218	
backend/algorithms/image_ops.py	op_find_contours	228	
backend/algorithms/logic_ops.py	_to_number	17	
backend/algorithms/logic_ops.py	_to_bool	27	
backend/algorithms/logic_ops.py	_parse_constant	37	
backend/algorithms/logic_ops.py	_scalar_value	46	确保逻辑比较接收到的是单个值，而不是图像或数组。
backend/algorithms/logic_ops.py	_comparison_number	61	
backend/algorithms/logic_ops.py	_compare	72	
backend/algorithms/logic_ops.py	_update	93	
backend/algorithms/logic_ops.py	op_logic_constant	108	
backend/algorithms/logic_ops.py	op_logic_compare	117	
backend/algorithms/logic_ops.py	op_logic_boolean	132	
backend/algorithms/logic_ops.py	op_logic_not	156	
backend/algorithms/logic_ops.py	op_logic_if_else	166	
backend/algorithms/logic_ops.py	op_logic_for_loop	186	
backend/algorithms/logic_ops.py	op_logic_while_loop	213	
backend/algorithms/plugin_loader.py	class CustomOperatorDefinition	15	
backend/algorithms/plugin_loader.py	_definition_from_module	24	
backend/algorithms/plugin_loader.py	load_custom_operators	63	自动加载 backend/custom_operators 下的插件。
backend/algorithms/registry.py	_append_catalog_item	94	
backend/algorithms/types.py	class ImageResolver	8	
backend/algorithms/types.py	class ExecutionContext	15	单次节点执行上下文。
backend/api/dependencies.py	get_image_store	6	

文件	函数/类	行	职责/文档
backend/api/routes/export_script.py	export_script	11	
backend/api/routes/operators.py	list_operators	9	
backend/api/routes/upload.py	upload_image	12	
backend/api/routes/workflow.py	execute_workflow	13	
backend/custom_operators/_template.py	op_my_operator	11	
backend/custom_operators/_template.py	compile_script	34	可选：支持“导出为独立算法脚本”。
backend/custom_operators/mean_brightness.py	op_mean_brightness	11	
backend/custom_operators/mean_brightness.py	compile_script	43	纯算法脚本导出扩展点。
backend/main.py	create_app	10	
backend/models/workflow.py	class NodePosition	8	
backend/models/workflow.py	class WorkflowNode	13	
backend/models/workflow.py	class WorkflowEdge	21	
backend/models/workflow.py	class WorkflowRequest	29	
backend/services/image_store.py	class StoredImage	12	
backend/services/image_store.py	class ImageStore	19	负责上传图片兼容存储，以及服务器本地图片路径解析。
backend/services/script_exporter.py	_normalize_type	13	
backend/services/script_exporter.py	_safe_identifier	28	
backend/services/script_exporter.py	_statement	35	
backend/services/script_exporter.py	_expression	39	
backend/services/script_exporter.py	_gray_expression	43	
backend/services/script_exporter.py	_bgr_expression	51	
backend/services/script_exporter.py	_odd	59	
backend/services/script_exporter.py	_integer	68	
backend/services/script_exporter.py	_floating	75	
backend/services/script_exporter.py	_compile_operator	82	Return statements, result expression variable placeholder, and output color space.
backend/services/script_exporter.py	_literal_constant	245	
backend/services/script_exporter.py	_logic_compare_expression	256	
backend/services/script_exporter.py	_logic_update_expression	263	
backend/services/script_exporter.py	_compile_logic_operator	273	
backend/services/script_exporter.py	generate_algorithm_script	356	
backend/services/script_exporter.py	generate_standalone_script	530	
backend/services/workflow_engine.py	class WorkflowExecutionError	19	工作流节点执行失败，并携带已经完成的执行现场。
backend/services/workflow_engine.py	image_to_data_url	79	
backend/services/workflow_engine.py	_json_safe_value	96	把节点结果限制为可安全返回给浏览器的 JSON 数据。
backend/services/workflow_engine.py	topological_sort	128	
backend/services/workflow_engine.py	_merge_data	158	

文件	函数/类	行	职责/文档
backend/services/workflow_engine.py	_prepare_graph	165	
backend/services/workflow_engine.py	_execute_once	176	
backend/services/workflow_engine.py	_yes	303	
backend/services/workflow_engine.py	_integer	309	
backend/services/workflow_engine.py	_parse_extensions	316	
backend/services/workflow_engine.py	_flatten_metrics	329	
backend/services/workflow_engine.py	_safe_csv_name	347	
backend/services/workflow_engine.py	_output_suffix	354	
backend/services/workflow_engine.py	_save_result_image	365	
backend/services/workflow_engine.py	_write_csv	379	
backend/services/workflow_engine.py	_validate_batch_workflow	396	
backend/services/workflow_engine.py	run_batch_workflow	419	
backend/services/workflow_engine.py	run_workflow	534	

B.2 前端主要函数分组

模块职责	函数
多工作流与存储	emptyWorkflow, createWorkflowDocument, persistWorkspace, restoreWorkflowDocument, switchWorkflow, saveWorkflowLocal
节点库	initOperators, renderOperatorPanel, renderCategoryOperatorPanel, collapse/expand groups
画布视图	applyTransform, screenToWorld, setZoom, resetView
节点渲染	renderNodes, renderNodeToolbarHtml, renderNodeParameterControls, attachNodeEvents
端口与连线	nodeInputPorts, nodeOutputPorts, portsAreCompatible, startConnect, finishConnect, renderEdges
执行与预览	buildWorkflow, runWorkflow, renderPreviews, renderExecutionFailureSummary, renderBatchSummary
图片交互	downloadNodePreview, openImageLightbox, closeImageLightbox
导航视图	computeMinimapTransform, updateMinimap, navigateFromMinimap

附录 C：常用修改路径与检查清单

需求	应修改的位置
新增自定义图像/测量算子	custom_operators 新文件 → 运行测试 → /api/operators 检查；需要导出时实现 script_compiler。
新增核心图像算子	image_ops → metadata → registry → script_exporter → tests。
新增逻辑节点	logic_ops → metadata 多端口 → registry → script_exporter → 前端类型兼容 → tests。
修改参数控件	metadata.param_schema；通用前端控件通常无需改。
新增 API	api/routes 新模块 → api/router.py 注册 → api.js 封装。

需求	应修改的位置
修改结果展示	workflow_engine preview 协议 → app.js renderNodePreviewHtml/renderPreviews → CSS → 前端测试。
修改批量输出	workflow_engine.run_batch_workflow、 _flatten_metrics、_write_csv。
修改路径规则	config.py + image_store.py，并增加安全测试。
修改多 workflow 保存	app.js workflowDocuments、localStorage payload 和迁移逻辑。
修改导航视图	app.js computeMinimapTransform/updateMinimap/navigateFromMinimap。

C.2 自定义算子提交前检查

- 插件文件名不以下划线开头；type_name 唯一。
- 输入端口和返回键一致。
- 颜色空间明确，灰度输出标记 GRAY。
- metrics 中没有 np.ndarray 或不可序列化对象。
- NumPy 数值转换为 Python 标量。
- 批量流程下可重复执行，不依赖上一次调用状态。
- 参数有默认值、schema、范围验证和错误提示。
- 增加单图、错误输入、批量 CSV 和脚本导出测试。

结语

当前项目已经具备清晰的后端分层、元数据驱动节点、插件自动注册、单图/批量执行、逻辑数据流、部分错误结果保留和 AST 导出能力。后续维护的关键是保持四个协议一致：工作流 JSON、算子返回字典、metadata 端口声明、前端预览协议。任何新功能都应同时考虑网页运行、批量数据汇总、错误现场和独立脚本导出四条链路。

维护原则：先写或更新测试，再修改实现；每次交付从完整基线复制并在全新目录解压运行。